



Control Flows



Why do we need control flow?

Why do we need control flow?

 A traffic light checks conditions: **red** → stop, **green** → go, **yellow** → slow down. Your Python code does the same thing!

Game

Is health > 0?

→ **Keep playing**

→ **Game over**

Banking

Is balance enough?

→ **Allow transfer**

→ **Decline**

Weather App

Is temp < 10°C?

→ **Show coat icon**

→ **Show sun icon**



What is control flow?

What is control flow?

Control flow structures allow you to dictate the order in which **statements** execute in your program.

Python manages control flow using 2 core structures

1

Conditional Statements

if / elif / else — branch based on True/False checks.

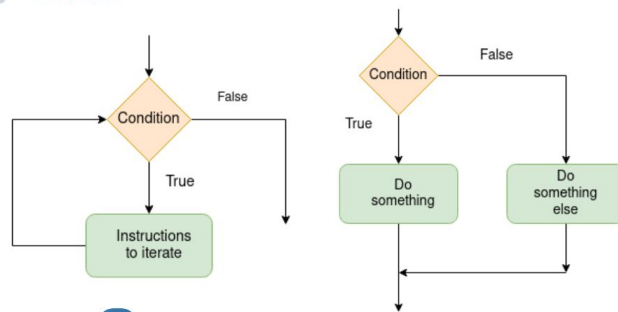
2

Loop & Transfer Statements

Repeat code and control iteration (covered in a later lesson).



DATA SCIENCE
PARICHAY



Python Control flow

Conditional Statements



The `if` Statement

Control Flows: The `if` Statement

"If" statements let us control the order code runs in, based on conditions.

- **if** keyword — starts the decision check
- **condition** — an expression that is True or False
- **:** colon — REQUIRED at the end of the line
- **indented block** — only runs if condition is True

Code Sample



```
# Variable assignment
age = 18

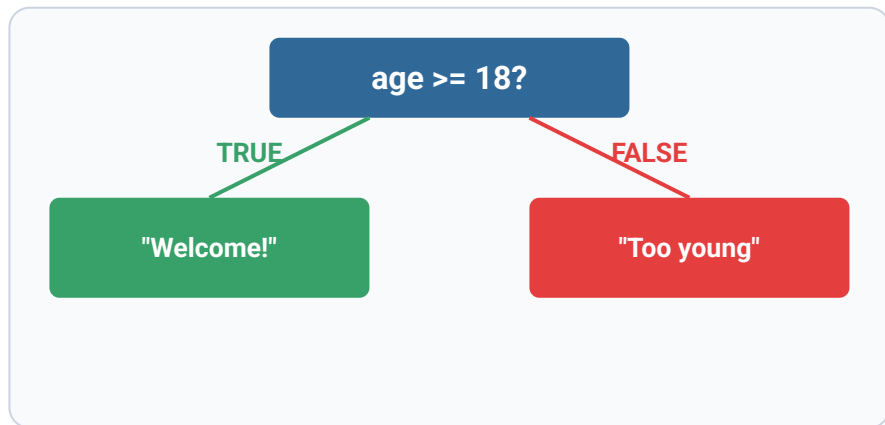
if age ≥ 18:
    print("You can vote!")
```




Control flow: **`else`** Statement - A Backup Plan

Control flow: `else` Statement - A Backup Plan

Program to check age:



Code visualizer:

- <https://programiz.pro/code-visualizer/python>
- <https://pythontutor.com/visualize.html#mode=display>
- <https://learn-code-with-vengleab.vercel.app/>

Code Sample

```
# Current status
age = 15

if age >= 18:
    print("Welcome!")
else:
    print("Sorry, too young")
```

execution order

L1

code	step 1/4	running line 1
1	>	age = 15
2		
3	if	age >= 18:
4		print("You can vote!")
5	else:	
6		print("Sorry, too young")

variables (changed in green)

(none yet)

printed output

Challenge: change age to 20 and re-run — what happens to the output?



Control flow: `elif` Handles Multiple Scenarios

Control flow: `elif` Handles Multiple Scenarios

Grading app:

score $\geq$ 90 ? → A — Excellent

score $\geq$ 80 ? → B — Good

score $\geq$ 70 ? → C — Pass

else (catch-all) → F — Fail

 **Insight:** Python checks conditions top-to-bottom and stops at the FIRST match.

Code Sample

```
score = 75

if score ≥ 90:
    grade = "A"
elif score ≥ 80:
    grade = "B"
elif score ≥ 70:
    grade = "C"
else:
    grade = "F"

print(f"Grade: {grade}")
```

Visualize code execution

execution order

L1

code step 1/7 running line 1

```
1 > score = 75
2
3 if score >= 90:
4     grade = "A"
5 elif score >= 80:
6     grade = "B"
7 elif score >= 70:
8     grade = "C"
9 else:
10    grade = "F"
11
12 print(f"Grade: {grade}")
```

variables (changed in green)

(none yet)

printed output



Test your knowledge

What Does This Code Print?

```
x = 7

if x > 10:
    print("Big")
elif x > 5:
    print("Medium")
else:
    print("Small")
```

A) Big

B) Medium

C) Small

D) Nothing – syntax error

What Does This Code Print?



```
x = 7

if x > 10:
    print("Big")
elif x > 5:
    print("Medium")
else:
    print("Small")
```

A) Big

B) Medium

C) Small

D) Nothing – syntax error

 **Insight:** Answer: B) Medium – x is 7, which is > 5 but NOT > 10, so elif matches first.

Nested if Statements

`Nested if` Statements Go Deeper Into Decisions

INDENTATION = NESTING LEVEL

Level 0 — outer if

Level 1 — inner if

Level 2 — inner else

● **Insight:** avoid nesting more than 2–3 levels deep. Consider combining conditions with `and` / `or` instead.

Code Example for login check

execution order

L1

code step 1/6 running line 1

```
1 > logged_in = True
2 is_admin = False
3
4 if logged_in:
5     if is_admin:
6         print("Admin panel")
7     else:
8         print("User dashboard")
9 else:
10    print("Please log in")
11
12 # Output: User dashboard
```

variables (changed in green)

(none yet)

printed output

Ternary Expressions

Ternary Expressions: Simple If-else Into One Line

😓 Before (4 lines)

```
if age ≥ 18:  
    label = "Adult"  
else:  
    label = "Minor"
```

😎 After (1 line)

```
label = "Adult" if age ≥ 18 else "Minor"
```



ANATOMY OF Ternary Expressions

value_if_true if **condition** else **value_if_false**

● **Insight:** use ternary for simple, readable assignments. For complex logic, stick with full if/elif/else.

Practices - Basic If-else

Positive Number & Even Number

TRY IT YOURSELF · LEVEL 1 · EXERCISES 1–2

1 Positive Number

- Ask user to input a **number**
- If **positive**, print `Positive`

```
ex01.py

number = float(input("Enter a number: "))

# TODO: check if number is positive
```

Expected output:

[1] number: 10 → Output: Positive
[2] number: -10 → Output:

2 Even Number

- Ask user to input an **integer**
- If **even**, print `Even`

```
ex02.py

num = int(input("Enter an integer: "))

# TODO: use % to check even
```

Expected output:

[1] num=10 → Output: even
[2] num= 9 → Output:

Adult Check & Pass or Fail

TRY IT YOURSELF · LEVEL 1 · EXERCISES 3–4

3 Adult Check

- Input `age`
- 18+ -> `Adult` else `Minor`

```
ex03.py

age = int(input("Enter age: "))

# TODO: compare age to 18
```

Expected output:

- [1] age= 18 → Output: Adult
- [2] age = 10 → Output: Minor
- [3] age= 19 → Output: Adult

4 Pass or Fail

- Input `score`
- `>= 50` -> `Pass`, else `Fail`

```
ex04.py

score = float(input("Enter score: "))

# TODO: compare score to 50
```

Expected output:

- [1] score =10 → Output: Fail
- [2] score = 50 → Output: Pass
- [3] score = 100 → Output: Pass

Bigger Number & Temperature

TRY IT YOURSELF · LEVEL 1 · EXERCISES 5-6

5 Bigger Number

- Ask user to Input **two** integers
- Print the **larger** one

```
ex05.py  
  
a = int(input("First number: "))  
b = int(input("Second number: "))  
  
# TODO: compare a and b
```

Expected output:

```
[1] a = 20, b = 20 → Output:  
[2] a = 20, b = 10 → Output: 20  
[3] a = 18, b = 19 → Output: 19
```

6 Temperature

- Ask user to Input **temperature**
- > 30 -> `Hot`, else `Cool`

```
ex06.py  
  
temp = float(input("Enter temperature: "))  
  
# TODO: compare temp to 30
```

Expected output:

```
[1] temp = 10 → Output: Cool  
[2] temp = 30 → Output: Cool  
[3] temp = 100 → Output: Hot
```


Divisible by 5 & Username Check

TRY IT YOURSELF · LEVEL 1 · EXERCISES 7–8

7 Divisible by 5

- Ask user to input a number
- Divisible by 5 -> `Yes`, else `No`

```
ex07.py

num = int(input("Enter a number: "))

# TODO: use % 5
```

Expected output:

- [1] num = 5 → Output: Yes
- [2] num = 10 → Output: Yes
- [3] num = 14 → Output: No

8 Username Check

- Ask user to input a **username**
- `admin` -> `Welcome Admin`, else `Access Denied`

```
ex08.py

username = input("Enter username: ")

# TODO: compare to "admin"
```

Expected output:

- [1] username = admin → Output: Welcome Admin
- [2] username = Admin → Output: Access Denied
- [3] username = 100 → Output: Hot

Practices - Tenerary

Ternary: Maximum & Ternary: Even/Odd

TRY IT YOURSELF · LEVEL 1 · EXERCISES 9–10

9 Ternary: Maximum

```
ex09.py

a = int(input("First: "))
b = int(input("Second: "))

# TODO: one-line ternary
result = ... # a if ... else b
```

- Use **only** a ternary operator
- Print the **larger** number

10 Ternary: Even/Odd

```
ex10.py

num = int(input("Enter integer: "))

# TODO: ternary -> "Even" or "Odd"
```

- Use a **ternary** operator
- Print `Even` or `Odd`

Practices - Banking and Shopping related

Low Balance Warning & Withdrawal Check

TRY IT YOURSELF · BANKING · EXERCISES 1-2

1 Low Balance Warning

- Ask user to input **balance** and **threshold**
- If balance is **below** threshold, print `Low balance warning`

ex01.py

```
balance = float(input("Enter balance: "))
threshold = float(input("Enter threshold: "))

# TODO: print a warning if balance is below
threshold
```

Expected output:

- [1] balance=45, threshold=100 → Output: Low balance warning
[2] balance=150, threshold=100 → Output: (nothing)

2 Withdrawal Check

- Ask user to input **balance** and **withdrawal**
- If withdrawal $\leq$ balance print `Approved`, else `Insufficient funds`

ex02.py

```
balance = float(input("Enter balance: "))
withdrawal = float(input("Enter withdrawal: "))

# TODO: print Approved or Insufficient funds
```

Expected output:

- [1] balance=200, withdrawal=250 → Output: Insufficient funds
[2] balance=200, withdrawal=150 → Output: Approved

Account Tier & Transfer Verification

TRY IT YOURSELF · BANKING · EXERCISES 3-4

3 Account Tier

- Ask user to input **balance**
- Print the matching tier: Platinum ≥ 10000 , Gold ≥ 5000 , Silver ≥ 1000 , else Basic

```
ex03.py

balance = float(input("Enter balance: "))

# TODO: classify into Platinum / Gold / Silver /
Basic
```

Expected output:

- [1] balance=7500 → Output: Gold
[2] balance=800 → Output: Basic

4 Transfer Verification

- First check if account is **verified**
- If verified, check amount against the **daily limit**

```
ex04.py

verified = input("Verified? (yes/no): ") == "yes"
amount = float(input("Enter amount: "))
daily_limit = 5000

# TODO: nested check: verified, then within daily
limit
```

Expected output:

- [1] verified=yes, amount=6000 → Output: Flagged for review
[2] verified=no, amount=3000 → Output: Account not verified

Free Shipping & Promo Code

TRY IT YOURSELF · SHOPPING · EXERCISES 1-2

5 Free Shipping

- Ask user to input **cart total**
- If \$50 or more, print `You qualify for free shipping!`

```
ex09.py

cart_total = float(input("Enter cart total: "))

# TODO: check if total qualifies for free shipping
```

Expected output:

- [1] cart_total=62 → Output: free shipping!
- [2] cart_total=30 → Output: (nothing)

6 Promo Code

- Ask user to input **code** and **price**
- Apply 10% discount only if code equals `SAVE10`

```
ex10.py

code = input("Enter promo code: ")
price = float(input("Enter price: "))

# TODO: apply 10% off if code == SAVE10
```

Expected output:

- [1] code=SAVE10, price=80 → Output: 72.0
- [2] code=X, price=80 → Output: 80

Discount Tiers & Membership Discount

TRY IT YOURSELF · SHOPPING · EXERCISES 3-4

7 Discount Tiers

- Ask user to input **amount spent**
- Print discount tier: 20% ≥ 200 , 10% ≥ 100 , 5% ≥ 50 , else none

ex11.py

```
spent = float(input("Enter amount spent: "))

# TODO: print discount tier
```

Expected output:

- [1] spent=120 → Output: 10% off
[2] spent=40 → Output: No discount

8 Membership Discount

- First check if customer **is a member**
- If member, check total against loyalty threshold (\$100)

ex12.py

```
is_member = input("Member? (yes/no): ") == "yes"
total = float(input("Enter total: "))

# TODO: nested check: member, then loyalty bonus
```

Expected output:

- [1] is_member=yes, total=150 → Output: loyalty discount
[2] is_member=no, total=200 → Output: No discount